

PROGRAMACIÓN DE SERVICIOS Y PROCESOS

UT4. GENERACIÓN DE SERVICIOS EN RED

GUILLERMO PALAZÓN CANO

LICENCIA [CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/)



- ▶ Las redes de ordenadores (y de otros dispositivos) son una fuente increíble de oportunidades para desarrollar aplicaciones.
 - ▶ Crear programas que se ejecuten de manera distribuida entre varios ordenadores.
 - ▶ Obtener información de páginas web y servicios web.
 - ▶ Transferir ficheros.
 - ▶ Enviar correos electrónicos.
 - ▶ Etc.
- ▶ Los servicios en red son programas auxiliares utilizados en un sistema para gestionar una colección de recursos y prestar su funcionalidad a los usuarios y aplicaciones.
 - ▶ El acceso a los servicios está formado por el conjunto de operaciones que ofrece. Por ejemplo, un servicio de archivos ofrece operaciones de lectura, escritura y borrado de ficheros.

- ▶ Los servicios de Internet siguen el modelo cliente-servidor y utilizan protocolos abiertos de la capa de aplicación del modelo TCP/IP (generalmente), por lo que pueden ser empleados y programados desde cualquier lenguaje de programación.
 - ▶ La mayor parte de los lenguajes disponen de clases y librerías adecuadas para poder utilizar estos de manera sencilla y adecuada (Existen multitud de librerías para trabajar con los servicios más comunes)
- ▶ Existen muchos protocolos estándares de nivel aplicación, basados en los protocolos de transporte UDP y TCP, que proporcionan servicios diversos a las aplicaciones.
 - ▶ En esta unidad nos vamos a centrar en el servicio FTP (transferencia de archivos), servicio de correo electrónico y la comunicación mediante HTTP (aunque no son los únicos, existen otros como TELNET, SSH, DNS, etc).

1. TRANSFERENCIA DE FICHEROS MEDIANTE FTP

- ▶ En Java, de manera nativa, es posible realizar transferencias de ficheros mediante este protocolo, pero es muy tedioso y en ocasiones difícil de hacerlo.
- ▶ La librería Apache Commons Net proporciona clases y utilidades para realizar cualquier operación sobre un servidor FTP o FTPS desde un cliente Java.
- ▶ Además, implementa muchos protocolos básicos de Internet.
 - ▶ Algunos de estos protocolos soportados por esta librería son: FTP/FTPS, FTP over HTTP (experimental), NNTP, SMTP(S), POP3(S), IMAP(S), Telnet, TFTP, Finger, Whois, rexec/rcmd/rlogin, Time (rdate) and Daytime, Echo, Discard o NTP/SNTP.
- ▶ La página oficial de esta librería y desde donde es posible su descarga es <https://commons.apache.org/proper/commons-net/>

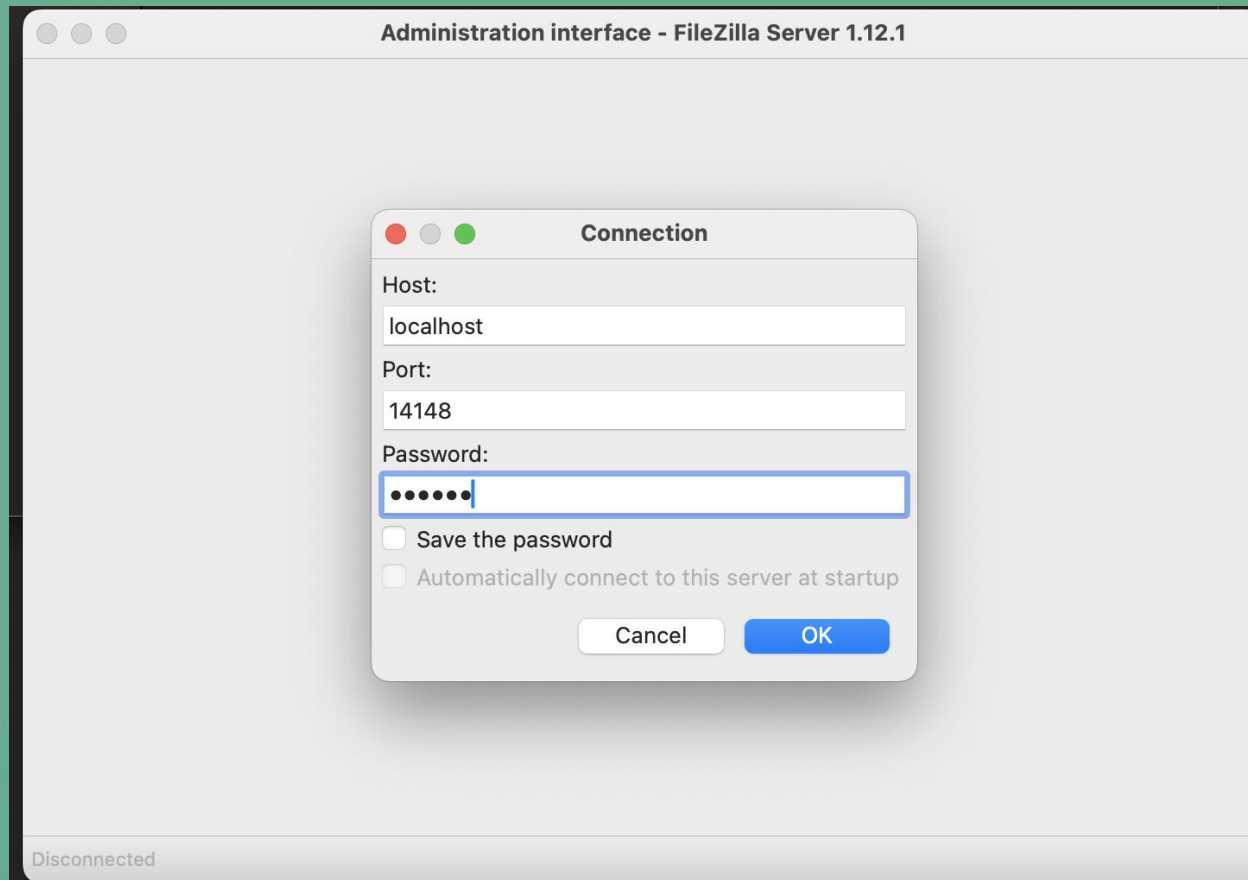
1.1. Servidor FTP

- ▶ Un servidor FTP es un software diseñado específicamente para la gestión y transferencia de archivos en una red. Su función es actuar como un "almacén" centralizado al que otros equipos (clientes) pueden conectarse para subir o descargar documentos, imágenes o código, utilizando el protocolo estándar FTP.
- ▶ Nuestra herramienta: FileZilla Server
 - ▶ Para nuestras prácticas, utilizaremos FileZilla Server. Es una de las opciones más populares y potentes dentro del mundo del software libre, ideal para aprender cómo funciona este servicio sin complicaciones innecesarias.



► Pasos iniciales:

- Instalación: El proceso es el clásico asistente "Siguiendo, Siguiendo". Es muy ligero y no requiere configuraciones complejas de sistema durante la instalación.
- Puesta en marcha: Una vez instalado, toda la gestión se realiza desde el FileZilla Server Interface. Desde esta consola podremos controlar quién se conecta, a qué carpetas tiene acceso y qué permisos tiene (solo lectura, escritura, borrado, etc.).
- Es importante que veáis el servidor FTP no solo como una carpeta compartida, sino como un servicio que debéis configurar correctamente para que vuestras futuras aplicaciones Java puedan interactuar con él de forma segura.



Administration interface - FileZilla Server 1.12.1		
Info	Type	Message
... Admin UI	Status	Successfully connected to server localhost:14148. Server'

- 
- ▶ Una vez que tenemos el servidor instalado, el siguiente paso es "darlo de alta" y configurar quién puede entrar y a qué carpetas. Para ello, abrimos el FileZilla Server Interface y vamos al menú Server > Configure.

▶ 1. Creación de Usuarios

- ▶ Dentro del panel de configuración, buscaremos la sección Users para definir quién tiene permiso para conectarse:
 - ▶ Pulsamos en "Add" y asignamos un nombre al usuario.
 - ▶ En el apartado de autenticación, seleccionamos "Require a password to login".
 - ▶ Introducimos la contraseña. Recordad que este será el usuario y clave que luego usaremos en nuestro código Java para loguearnos.

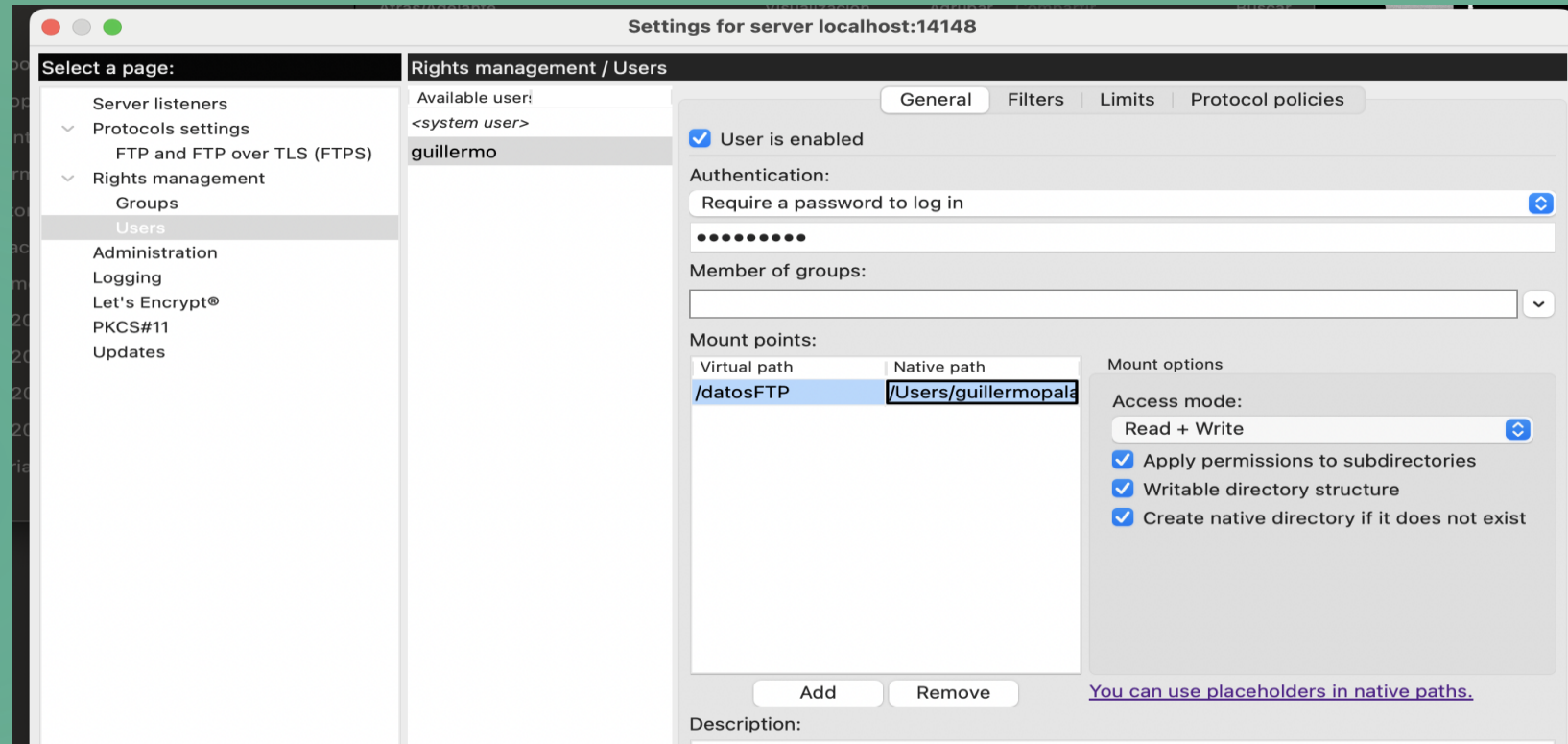
► 2. Puntos de Montaje (Mounting Points)

- Un servidor FTP no deja ver todo el disco duro por seguridad; solo muestra lo que nosotros decidamos. Para configurar esto, vamos a la sección Mounting Points:
 - Pulsamos en "Add" para crear una nueva ruta.
 - En el campo "Native path", seleccionaremos la carpeta real de nuestro ordenador que queremos compartir (por ejemplo, C:\ftp_datos si nuestro sistema es un SO Windows o similar en otro sistema operativo).
 - En el campo "Virtual path", pondremos cómo verá el usuario esa carpeta (normalmente se usa / para que sea la carpeta raíz al entrar).
- Es vital configurar correctamente los permisos de lectura y escritura en esta misma sección si queremos que nuestras aplicaciones puedan subir archivos y no solo descargarlos.
- Una vez que hayamos creado el usuario y el punto de montaje, tendremos que hacer clic en el botón "OK" para guardar los cambios.

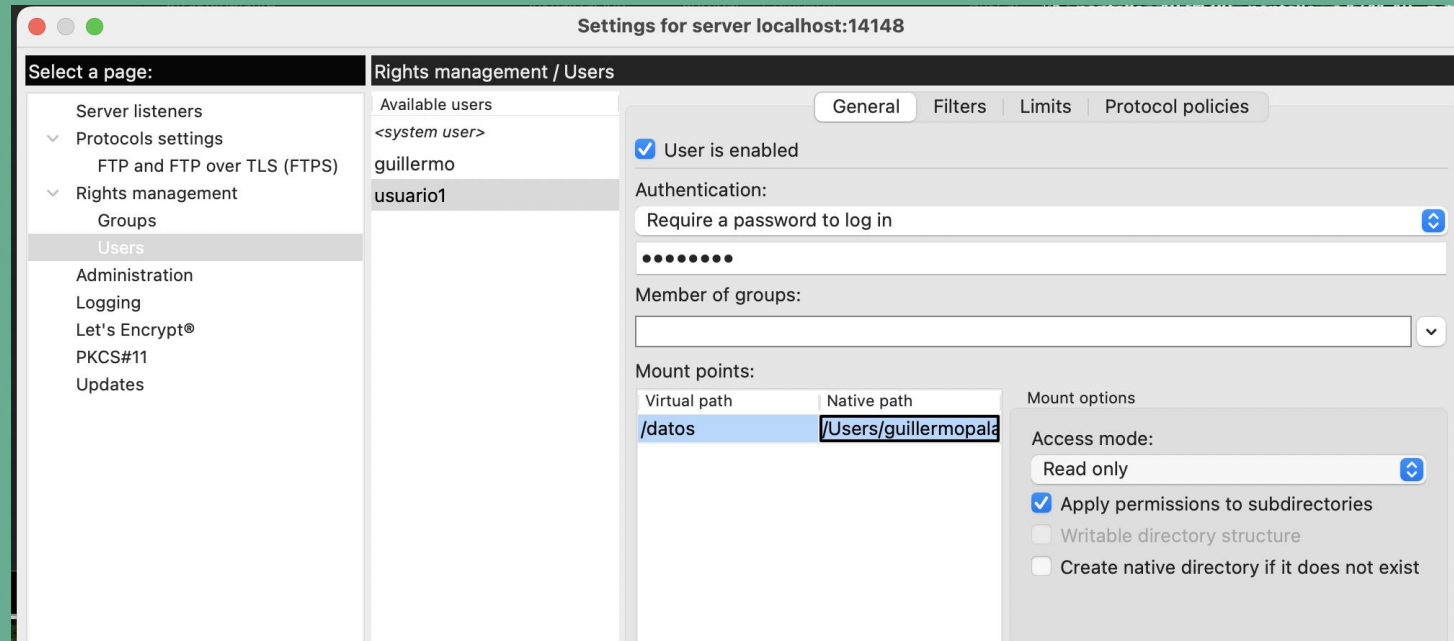
► Rutas Virtuales vs. Rutas Nativas

- Para que el servidor sea seguro y esté bien organizado, no mostramos la estructura real de carpetas de nuestro disco duro. En su lugar, utilizamos Alias o nombres ficticios.
 - Ruta Nativa (Native Path): Es la ubicación física y real en el disco duro del servidor. Es la ruta que entiende el sistema operativo. Ejemplo: C:\Users\Proyectos\Java\archivos_ftp
 - Ruta Virtual (Virtual Path): Es el nombre público o "apodo" que verá el cliente cuando se conecte. Ejemplo: /mis_datos
- ¿Por qué utilizar este sistema de "Alias"?
 - Seguridad: El cliente nunca sabe en qué carpeta real o en qué unidad de disco (C:, D:, etc.) están guardados los archivos
 - Organización: Puedes tener archivos en carpetas muy profundas o incluso en diferentes discos duros, pero para el usuario aparecerán todos juntos y ordenados como si estuvieran en una misma carpeta.
- En resumen: Es como un acceso directo en el escritorio. El archivo real puede estar escondido en una carpeta muy profunda (Ruta Nativa), pero tú lo abres desde un icono que tienes a mano con un nombre sencillo (Ruta Virtual).

- ▶ Creo el usuario guillermo, con ruta virtual datosFTP y que tiene permisos de lectura y escritura.
- ▶ He puesto como contraseña también el usuario guillermo (no es lo más recomendable)



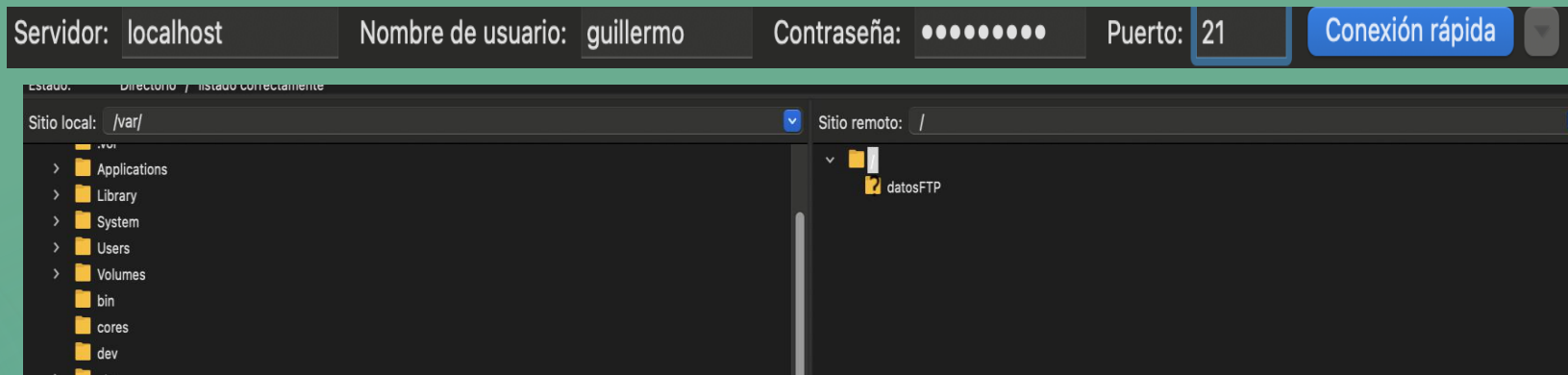
- Creo otro usuario, usuario1 que solamente tiene permisos de lectura y he cambiado la ruta que el usuario va a ver (aunque realmente acceden al mismo directorio). En este caso el usuario verá /datos



1.2. Cliente FTP

- ▶ En esta unidad, el principal interés reside en revisar las clases y métodos principales del paquete `org.apache.commons.net.ftp` que nos van a permitir conectarnos y autenticarnos con el servidor, cambiarnos de directorios, enviar archivos al servidor, descargar archivos del servidor, etc.
- ▶ En cualquier caso, para probar antes de comenzar con estas librerías vamos a descargarnos el cliente FTP de Filezilla y comprobar que efectivamente tenemos acceso con estos usuarios creados y que podemos hacer las acciones que realmente se han definido para los mismos.
- ▶ Cliente: <https://filezilla-project.org/download.php?type=client>

- ▶ Pruebo la conexión con el usuario guillermo en el puerto de FTP (puerto 21, ya que el puerto que hemos visto en capturas anteriores (14148) es solo para administración).



- ▶ Pruebo a copiar un fichero y como tengo permisos de escritura permite subir el mismo.
 - ▶ Se busca la ruta en el sitio local, se selecciona el fichero o carpeta y se le da a subir.

Sitio local: /Users/guillermopalazoncano/Python/

Prueba

Public

Python

VirtualBox VMs

basex

carpeta sin título

eclipse

eclipse-workspace

pruebas

s1851

sge05

user

Volumes

bin

cores

Nombre de archivo	Tamaño de arc	Tipo de archivo	Última modificación
cano.xml	2.008	Microsoft Word X..	03/02/2025 00:...
datosT001.txt	15	txt-archivo	18/01/2025 09:5...
productos.json	3.362	JSON File	03/02/2025 17:4...
pruebas.py		Script	18/01/2025 08:4...
pruebas2.p		Script	18/01/2025 09:1...
pruebas3.p		Script	18/01/2025 09:4...
pruebas4.p		Script	18/01/2025 09:5...
salida.txt		ivo	18/01/2025 09:5...

1 archivo selec

Subir

Añadir archivos a la cola

Abrir

Edición

Crear directorio

Crear directorio y entrar en él

Sitio remoto: /datosFTP

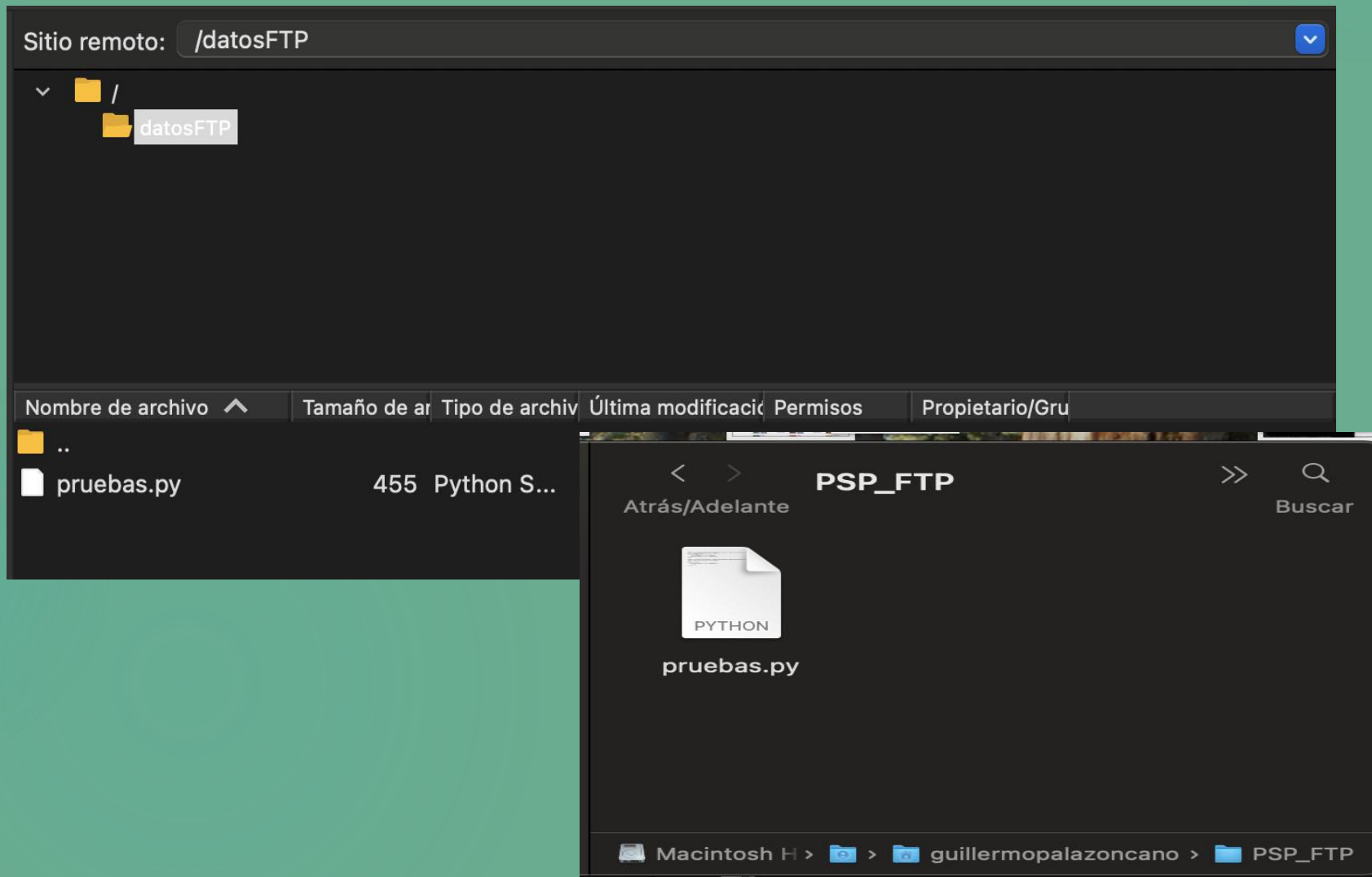
/

datosFTP

Nombre de archivo	Tamaño de ar	Tipo de archiv	Última modificació	Permisos	Propietario/Gru
..					

Listado del directorio vacío

Directorio vacío.



- ▶ Tratamos de subir ahora un archivo con el usuario usuario1
- ▶ Podemos ver es que la ruta virtual ahora no es datosFTP sino que únicamente será datos

Estado: Comenzando la subida de /Users/guillermopalazoncano/Python/pruebas4.py
Comando: EPSV
Respuesta: 229 Entering Extended Passive Mode (|||61462|)
Comando: STOR pruebas4.py
Respuesta: 550 Permission denied
Error: Error crítico de transferencia de archivo
Estado: Desconectado del servidor

Sitio local: /Users/guillermopalazoncano/Python/ Sitio remoto: /datos

Nombre de archivo Tamaño de archivo Tipo de archivo Última modificación

pruebas.py	455	Python Script	18/01/2025 08:4...
pruebas2.py	310	Python Script	18/01/2025 09:1...
pruebas3.py	586	Python Script	18/01/2025 09:4...
pruebas4.py	477	Python Script	18/01/2025 09:5...
salida.txt	12	txt-archivo	18/01/2025 09:5...

Nombre de archivo Tamaño de archivo Tipo de archivo Última modificación Permisos Propietario/Gru

..					
pruebas.py	455	Python S...	21/01/2026 2...		

Servidor/Archivo local	Dirección	Archivo remoto	Tamaño	Prioridad	Hora	Razón
usuario1@localhost						
/Users/guillermopalazo.	-->	/datos/pruebas4.py	477	Normal		No se pudo comenzar ..

1.3. Conectar y autenticar

- ▶ Para interactuar con un servidor FTP utilizaremos la clase `FTPClient` ya que vamos a hacer la conexión en texto plano (sería más profesional utilizar `FTPSCient` y hacer una conexión con TLS, pero para los contenidos de esta materia es suficiente con `FTPClient`) de la librería Apache Commons Net
- ▶ El proceso para establecer una comunicación sigue tres pasos fundamentales:
 - ▶ 1. El Objeto y la Conexión Lo primero es crear un objeto de `FTPClient` y llamar al método `connect()`. Aquí es donde le decimos los datos de la conexión (servidor y puerto) → Método: `cliente.connect(servidor, puerto);`
 - ▶ 2. A continuación, tendremos que comprobar que la conexión se ha establecido bien, para ello tendremos llamamos a `getReplyCode` y comprobar que el valor devuelto por este método es positivo. Si no lo es, desconectamos para evitar errores en cascada.
 - ▶ Realmente se usa `FTPReply.isPositiveCompletion(reply)` que devuelve `true` si el servidor ha aceptado correctamente la operación y `false` si algo ha ido mal.
 - ▶ 3. El Login (Autenticación) Una vez que sabemos que la conexión está ok, nos identificamos, enviando los datos del usuario que configuramos previamente en el servidor (usuario y contraseña) → Método: `cliente.login(usuario, password);`

- Puesto que vamos a trabajar en texto plano tenemos que cambiar dos cuestiones en la seguridad de FileZilla Server (la segunda se debe realizar para cada usuario)

The image displays three screenshots of the FileZilla Server settings window, titled "Settings for server localhost:14148".

Top Screenshot: Server listeners
The "Server listeners" tab is selected. A table lists the listening addresses and ports. The "Protocol" column for both entries is set to "Explicit FTP over TLS and insecure plain FTP".

Address	Port	Protocol
0.0.0.0	21	Explicit FTP over TLS and insecure plain FTP
::	21	Explicit FTP over TLS and insecure plain FTP

Middle Screenshot: Rights management / Users
The "Rights management / Users" tab is selected. The "Protocol policies" sub-tab is active. The settings for "guillermo" are shown.

Available users	General	Filters	Limits	Protocol policies
<system user>				
guillermo				FTP authentication (insecure): Enabled – overrides groups policies
usuario1				FTPS (FTP over TLS) authentication: Disabled – overrides groups policies

Bottom Screenshot: Rights management / Users
The "Rights management / Users" tab is selected. The "Protocol policies" sub-tab is active. The settings for "usuario1" are shown.

Available users	General	Filters	Limits	Protocol policies
<system user>				
guillermo				FTP authentication (insecure): Enabled – overrides groups policies
usuario1				FTPS (FTP over TLS) authentication: Disabled – overrides groups policies

- ▶ Antes de empezar a escribir en el código debemos añadir la librería en el pom.xml (proyectos Maven)

```
<dependencies>
  <!-- Source: https://mvnrepository.com/artifact/commons-net/commons-net -->
  <dependency>
    <groupId>commons-net</groupId>
    <artifactId>commons-net</artifactId>
    <version>3.12.0</version>
    <scope>compile</scope>
  </dependency>
</dependencies>
```

```
public class ClienteFTP {
    public static void main(String[] args) {
        String servidor = "localhost";
        int puerto = 21;
        String usuario = "guillermo";
        String password = "guillermo";

        FTPClient ftp = new FTPClient();

        try {
            ftp.connect(servidor, puerto);

            int replyCode = ftp.getReplyCode();
            if (!FTPReply.isPositiveCompletion(replyCode)) {
                ftp.disconnect();
                throw new IOException("Error al conectarnos al servidor FTP");
            }

            boolean login = ftp.login(usuario, password);
            System.out.println("¿Conectado? " + login);
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
}
```

¿Conectado?true

1.4. Ocultación contraseñas

- ▶ Aunque no es objeto de esta materia, tener en nuestro código en texto la contraseña de un usuario no parece lo más seguro.
- ▶ Cualquiera que tenga el código de nuestro proyecto podría saberla y ya si lo subimos a mecanismos de control de versiones como Git el riesgo es aún mucho peor.
- ▶ Existen otras opciones mucho más potentes que las que vamos a ver a continuación, pero estas, al menos, nos evitan que esta contraseña quede expuesta en Git (primera solución) y accesible si alguien tiene el código de nuestro proyecto (ambas soluciones). Estas son:
 - ▶ Fichero de configuración externo (.properties)
 - ▶ Variables de entorno

1.4.1. Fichero de configuración .properties

- ▶ Es una solución especialmente usada en soluciones clásicas.
- ▶ Los datos de la conexión se "guardan" en un fichero distinto, el cual NO se sube al repositorio (se incluye en el .gitignore)
- ▶ Dentro de la carpeta src/main/resources crearemos el fichero ftp.properties y en el mismo añadiremos los datos necesarios
 - ▶ Este fichero siempre se le proporcionan los datos de la forma clave=valor
 - ▶ Si se quieren añadir comentarios con #

```
.gitignore x
1 # Fichero de configuración FTP con contraseñas
2 src/main/resources/ftp.properties
```

```
ftp.properties x
1 ftp.server=localhost
2 ftp.port=21
3 ftp.user=guillermo
4 ftp.password=guillermo
```

- ▶ Como los ficheros de configuración forman parte del proyecto, Maven los coloca en el classpath.
- ▶ Para leerlos correctamente usamos el ClassLoader, que es quien sabe dónde están esos recursos en tiempo de ejecución.
 - ▶ Necesitamos una clase para poder acceder a ClassLoader, se suele poner por claridad en el código la clase actual en la que se encuentra el código.
- ▶ `getResourceAsStream` nos devuelve un flujo de datos, que cargamos con la clase `Properties`.
 - ▶ El objeto `Properties` almacenará los pares clave = valor del fichero .
- ▶ Con el método `config.getProperty("valor_clave");` obtendremos cada uno de estos valores.
 - ▶ `config.getProperty("ftp.user");`

```
public class ClienteFTPProperties {
    public static void main(String[] args) {
        try {
            Properties config = new Properties();
            InputStream input = ClienteFTPProperties.class.getClassLoader().getResourceAsStream("ftp.properties");

            if (input == null) {
                throw new RuntimeException("No se encuentra el fichero ftp.properties");
            }

            config.load(input);

            String usuario = config.getProperty("ftp.user");
            String servidor = config.getProperty("ftp.server");
            int puerto = Integer.parseInt(config.getProperty("ftp.port"));
            String password = config.getProperty("ftp.password");

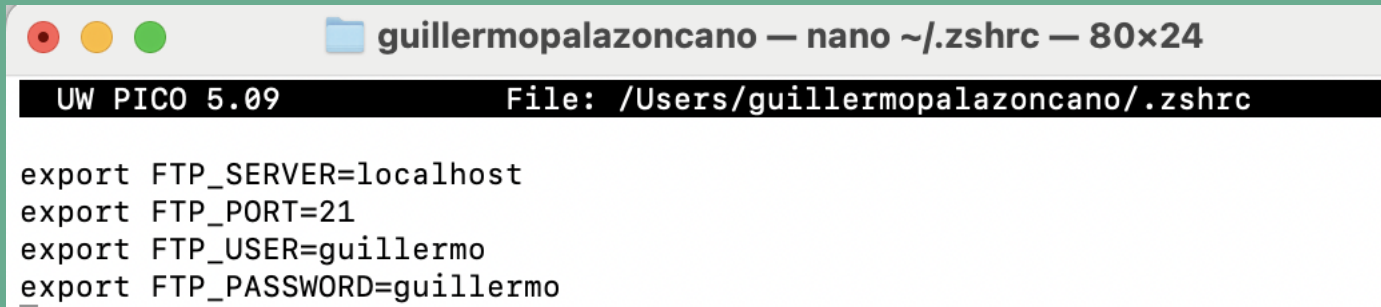
            FTPClient ftp = new FTPClient();
            ftp.connect(servidor, puerto);
            int replyCode = ftp.getReplyCode();
            if (!FTPReply.isPositiveCompletion(replyCode)) {
                ftp.disconnect();
                throw new IOException("Error al conectarnos al servidor FTP");
            }

            boolean login = ftp.login(usuario, password);
            System.out.println("¿Conectado?" + login);
        } catch (IOException e) {
            throw new RuntimeException("Error cargando configuración", e);
        }
    }
}
```

1.4.2. Variables de entorno

- ▶ Aunque es cierto que hemos protegido en parte nuestras contraseñas y podemos subir el proyecto a Git sin miedo a que estas estén accesibles, la persona que tenga el código de nuestro proyecto en su totalidad sí tendrá acceso al fichero `ftp.properties` y, por tanto, acceso a la contraseña de nuestro servidor.
- ▶ Podemos pasar al siguiente nivel de buenas prácticas y hacer uso de variables de entorno.
 - ▶ No se suben a Git porque no están en el proyecto.
- ▶ Una variable de entorno es un valor:
 - ▶ Definido fuera del código
 - ▶ Asociado al sistema operativo
 - ▶ Accesible por cualquier programa que se ejecute en él

- ▶ Puesto que las variables de entorno están asociadas al sistema operativo el proceso de crearlas es diferente según el SO.
- ▶ En Windows se pueden hacer de manera gráfica y, desde línea de comandos (CMD) de manera sencilla (no se pone el igual =)
 - ▶ setx FTP_SERVER localhost
 - ▶ setx FTP_PORT 21
 - ▶ setx FTP_USER guillermo
 - ▶ setx FTP_PASSWORD guillermo
- ▶ En UNIX (MacOS o Linux) si queremos que sean permanentes debemos añadirlas en el fichero .zshrc
 - ▶ Abrimos el fichero **nano ~/.zshrc**
 - ▶ Se añade cada variable en una línea distinta con el comando EXPORT
 - ▶ export NOMBRE_VARIABLE="valor_de_la_variable"
 - ▶ Se guarda el fichero y para cargar ya estas variables usamos **source ~/.zshrc**



```
export FTP_SERVER=localhost
export FTP_PORT=21
export FTP_USER=guillermo
export FTP_PASSWORD=guillermo
```

```
guillermopalazoncano@MacBook-Pro-de-Guillermo ~ % nano ~/.zshrc
guillermopalazoncano@MacBook-Pro-de-Guillermo ~ % source ~/.zshrc
[guillermopalazoncano@MacBook-Pro-de-Guillermo ~ % echo $FTP_USER
guillermo
```

- ▶ **MUY IMPORTANTE**: Es probable que tras establecer las variables de entorno debas reiniciar el IDE que estés utilizando para que este sea capaz de acceder al valor de las mismas.
- ▶ Una vez que las variables de entorno están creadas, en Java se leen de manera muy sencilla con el método

`System.getenv("NOMBRE_VARIABLE");`

```
String servidor = System.getenv(name: "FTP_SERVER");
int puerto = Integer.parseInt(System.getenv(name: "FTP_PORT"));
String usuario = System.getenv(name: "FTP_USER");
String password = System.getenv(name: "FTP_PASSWORD");

FTPClient ftp = new FTPClient();
ftp.connect(servidor, puerto);
```


1.5. Gestión de directorios

► **Gestión de Directorios: Local vs. Remoto**

- Al programar un cliente FTP, trabajamos en dos "mundos" simultáneos como ya hemos podido ver con nuestro cliente Filezilla
- Es fundamental no perder de vista en qué lado estamos operando en cada momento:
 - Directorio Local: Es la carpeta de nuestro propio ordenador (el cliente). Para movernos por aquí o seleccionar qué archivos queremos subir, utilizamos las clases estándar de Java que ya habéis visto en la materia de Acceso a Datos, como puede ser File o Path.
 - Directorio Remoto: Es la carpeta dentro del servidor FTP. Para trabajar en este entorno, necesitamos los métodos específicos de la clase FTPClient o FTPSClient según se haya establecido la conexión.

► Navegación y Control en el Servidor

- Para gestionar el "Directorio de Trabajo" en el servidor, contamos con varios métodos muy intuitivos:
 - **printWorkingDirectory()** nos devuelve una cadena con la ruta actual en el servidor (equivalente al comando pwd de Linux o cd en Windows).
 - Para cambiar de carpeta en el servidor, usamos **changeWorkingDirectory("nombre_carpeta")**.
 - Tenemos dos formas de ver el contenido del directorio de trabajo:
 - **listNames()**: Nos devuelve un array de Strings solo con los nombres de los archivos. Es rápido y sencillo.
 - **listFiles()**: Es mucho más potente, ya que devuelve un array de objetos FTPFile. Con ellos podemos consultar detalles como el tamaño del archivo, la fecha de creación o si es un directorio o un archivo.

- ▶ Antes de empezar con las pruebas de código he creado carpetas y he añadido ficheros en el “servidor” para que las pruebas nos muestren más información cuando las vayamos realizando. Este es el estado actual del servidor con 2 ficheros en la carpeta raíz, cuatro carpetas, teniendo cada carpeta uno o dos ficheros más.
- ▶ El código del apartado anterior (conexión) lo hemos encapsulado en un método que devolverá el objeto FTPClient (o null en caso de que no se consiga la conexión).
 - ▶ **IMPORTANTE**: Añadimos una llamada al método `enterLocalPassiveMode()` que puede solucionar problemas de puertos con firewalls.

Nombre	
	cab.py
	documento1.md
▼ 	ejercicios
	pruebas3.py
▼ 	examen
	documento4.md
	documento5.md
▼ 	fotos
	documento2.md
▼ 	trabajos
	documento3.md

```
private static FTPClient conexion() throws IOException { 5 usages
    String servidor = System.getenv(name: "FTP_SERVER");
    int puerto = Integer.parseInt(System.getenv(name: "FTP_PORT"));
    String usuario = System.getenv(name: "FTP_USER");
    String password = System.getenv(name: "FTP_PASSWORD");

    FTPClient ftp = new FTPClient();
    ftp.connect(servidor, puerto);

    int reply = ftp.getReplyCode();
    if (!FTPReply.isPositiveCompletion(reply)) {
        ftp.disconnect();
        throw new IOException("No se pudo conectar al servidor FTP");
    }

    boolean login = ftp.login(usuario, password);
    System.out.println("Login: "+login);
    if (!login) {
        ftp.enterLocalPassiveMode();
        ftp.disconnect();
        return null;
    }
    return ftp;
}
```



▶ EJERCICIO

- ▶ Vamos a crear el código que nos muestre el directorio actual en remoto al conectarnos
 - ▶ Nos saldrá la única ruta virtual que tiene definida ese usuario
- ▶ Nos cambiaremos de directorio para ir a la ruta virtual que tenemos definida para el usuario guillermo y mostraremos todos los ficheros y directorios de dicho directorio de trabajo.
- ▶ Finalmente, mostraremos el directorio de trabajo local (ruta proyecto) y los ficheros que se contienen en la misma en local.

```
try {
    FTPClient ftp = conexion();

    if (ftp != null) {
        System.out.println("\nDirectorio de trabajo remoto: " + ftp.printWorkingDirectory());
        // Mostrar el contenido del directorio de trabajo remoto actual
        for (String nombreFichero : ftp.listNames()) {
            System.out.println("\t"+nombreFichero);
        }
        // Cambiamos a la ruta virtual del usuario guillermo.
        ftp.changeWorkingDirectory(path: "/datosFTP");
        System.out.println("\nDirectorio trabajo remoto: " + ftp.printWorkingDirectory());

        FTPFile[] ficheros = ftp.listFiles();
        System.out.println("Ficheros/Directorios existentes en el servidor remoto:");
        for (FTPFile fichero : ficheros) {
            // No mostrar los ficheros ocultos.
            if(!fichero.getName().startsWith(".")) {
                System.out.println("\t" + fichero.getName());
            }
        }
    }
}
```

```
// Obtener el directorio de trabajo local actual
String directorioLocal = System.getProperty("user.dir");
System.out.println("\nDirectorio actual (local): "+directorioLocal);

// Mostrar el contenido del directorio de trabajo local actual
File file = new File(directorioLocal);
System.out.println("Ficheros/Directorios existentes en local:");
for (File fichero : file.listFiles()) {
    // No mostrar los ficheros ocultos.
    if(!fichero.getName().startsWith(".")) {
        System.out.println("\t" + fichero.getName());
    }
}

// Cambiar el directorio de trabajo local actual
//System.setProperty("user.dir", fileName);

ftp.logout();
ftp.disconnect();
}
} catch (IOException e) {
    e.printStackTrace();
}
```


Login: true

Directorio de trabajo remoto: /
datosFTP

Directorio trabajo remoto: /datosFTP

Ficheros/Directorios existentes en el servidor remoto:

- cab.py
- documento1.md
- trabajos
- examen
- fotos
- ejercicios

Directorio actual (local): /Users/guillermopalazoncano/[REDACTED]/PSP/ProyectosApuntes/UT04

Ficheros/Directorios existentes en local:

- target
- pom.xml
- src

1.6. Subida archivos al servidor

- ▶ No todos los archivos se tratan igual en una transferencia FTP. Dependiendo de lo que estemos enviando, debemos configurar nuestro FTPClient para que trabaje en uno de estos dos modos:
- ▶ **Modo ASCII (Texto)**
 - ▶ Se utiliza exclusivamente para archivos de texto plano (.txt, .html, .xml, archivos de código fuente, etc.).
 - ▶ ¿Qué hace? Adapta automáticamente los finales de línea al sistema operativo de destino. Por ejemplo, Windows usa \r\n (CRLF) y Linux usa \n (LF). El modo ASCII asegura que el texto se lea correctamente al llegar.
 - ▶ Método en Java: `cliente.setFileType(FTP.ASCII_FILE_TYPE);`
- ▶ **Modo Binario (Datos)**
 - ▶ Es el modo "tal cual". Se utiliza para cualquier archivo que no sea texto puro: imágenes, PDFs, ejecutables, archivos comprimidos (.zip) o música (si intentas subir un fichero de este tipo en modo texto es posible que se corrompa y no se pueda abrir después)
 - ▶ ¿Qué hace? Copia el archivo bit a bit, sin tocar ni un solo byte. No realiza ninguna conversión.
 - ▶ **Método en Java:** `cliente.setFileType(FTP.BINARY_FILE_TYPE);`
- ▶ En caso de duda el Modo Binario es el más seguro, ya que no altera el contenido (lo ideal es que el código detecte el tipo y cambie el modo según corresponda).

1.6.1. Modo ASCII

- ▶ Con el método `setFileType(FTP.ASCII_FILE_TYPE)` se indica al servidor que debe tratar el archivo como una secuencia de caracteres.
- ▶ Se abre el archivo local para lectura mediante un `InputStream` (lee el archivo local byte a byte o por bloques y lo envía de forma eficiente a través de la red)
- ▶ El método `storeFile()` es el que realmente ejecuta el comando del protocolo FTP para subir los datos al servidor.

```
public static boolean subirFicheroASCII(FTPClient ftp, String directorioFichero, 1 usage
                                     String nombreFichero) throws IOException {
    ftp.setFileType(FTP.ASCII_FILE_TYPE);
    // Recibiremos el fichero con ruta incluida.
    File fichero = new File(pathname: directorioFichero+"/"+nombreFichero);
    // Se nombrará igual en el Remoto
    String ficheroRemoto = nombreFichero;
    InputStream input = new FileInputStream(fichero);
    boolean upload = ftp.storeFile(ficheroRemoto, input);
    input.close();
    return upload;
}
```

```
private static void pruebaSubirFichero(){ 1 usage
    try {
        FTPClient ftp = conexion();

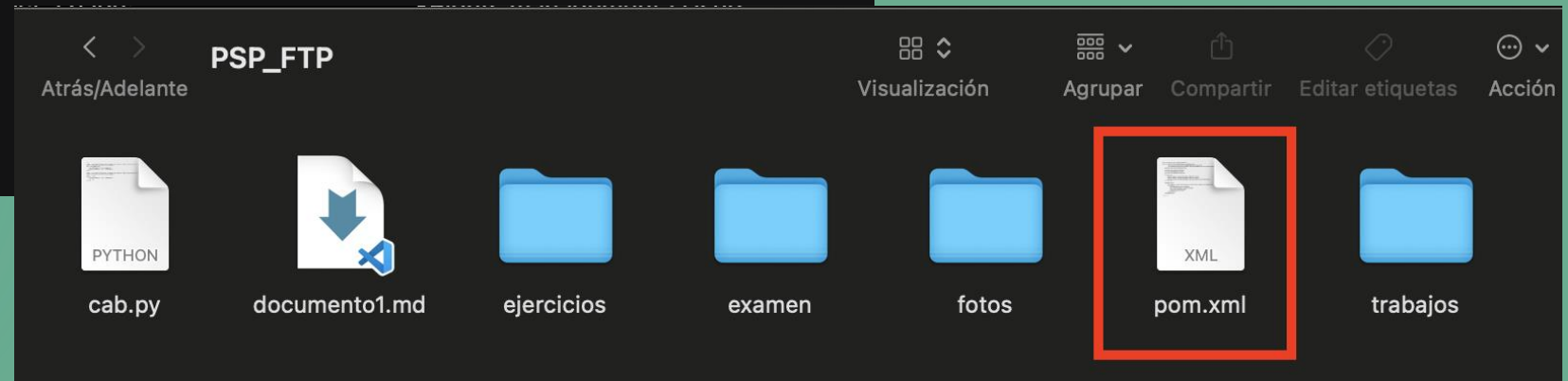
        if (ftp != null) {
            // Cambiamos a la ruta virtual del usuario guillermo.
            ftp.changeWorkingDirectory(path: "/datosFTP");

            // Obtener el directorio de trabajo local actual
            String directorioLocal = System.getProperty("user.dir");
            String nombreFichero = "pom.xml";

            // Subimos el fichero pom.xml
            boolean resultado = subirFicheroASCII(ftp, directorioLocal, nombreFichero);
            System.out.println("¿Se ha subido el fichero?" + resultado);

            ftp.logout();
            ftp.disconnect();
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

¿Se ha subido el fichero?true



1.6.2. Modo binario

- ▶ Ahora el programador es responsable de crear un buffer (en este caso de 4096 bytes) y de implementar un bucle while. Esto nos da un mayor control.
- ▶ `storeFileStream`: No sube el archivo directamente, sino que abre un "túnel" (un `OutputStream`) hacia el servidor. El archivo se va enviando conforme vas escribiendo en ese stream, en lugar de pasarle la responsabilidad completa a la librería.
- ▶ El comando `completePendingCommand` espera la respuesta definitiva del servidor que confirma que la transferencia fue exitosa.
 - ▶ Si se olvida esta línea, el programa podría quedarse bloqueado o la subida constar como fallida.

- He copiado una imagen en el directorio local (Imagen1.png) que será la que subiremos en modo binario.

```
public static boolean subirFicheroBinario(FTPClient ftp, String directorioFichero, 1
                                         String nombreFichero) throws IOException {

    ftp.setFileType(FTP.ASCII_FILE_TYPE);
    // Recibiremos el fichero con ruta incluida.
    File fichero = new File(pathname: directorioFichero+"/"+nombreFichero);
    // Se nombrará igual en el Remoto
    String ficheroRemoto = nombreFichero;
    InputStream input = new FileInputStream(fichero);
    OutputStream output = ftp.storeFileStream(ficheroRemoto);
    byte[] bytes = new byte[4096];
    int read = 0;

    while ((read = input.read(bytes)) != -1) {
        output.write(bytes, off: 0, read);
    }

    input.close();
    output.close();
    boolean upload = ftp.completePendingCommand();

    return upload;
}
```



```
try {
    FTPClient ftp = conexion();

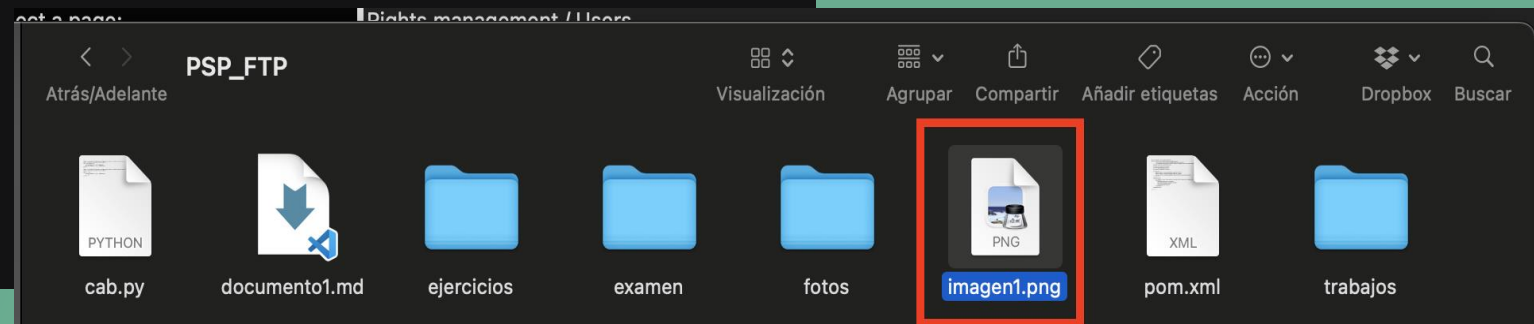
    if (ftp != null) {
        // Cambiamos a la ruta virtual del usuario guillermo.
        ftp.changeWorkingDirectory(path: "/datosFTP");

        // Obtener el directorio de trabajo local actual
        String directorioLocal = System.getProperty("user.dir");
        String nombreFichero = "imagen1.png";

        boolean resultado = subirFicheroBinario(ftp, directorioLocal, nombreFichero);
        System.out.println("¿Se ha subido el fichero?" + resultado);

        ftp.logout();
        ftp.disconnect();
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

¿Se ha subido el fichero?true



1.7. Descarga de archivos

- ▶ Al igual que para la subida, la descarga es diferente si el fichero a descargar es de tipo ASCII o de tipo Binario.
- ▶ Descarga tipo ASCII
 - ▶ A diferencia de la subida (donde usábamos un InputStream para leer), aquí usamos un FileOutputStream para **escribir** los datos que nos llegan de la red en el archivo local.
 - ▶ El método retrieveFile es el método principal. Se encarga de recibir los paquetes de datos del servidor y volcarlos automáticamente en el OutputStream
 - ▶ Para subir usamos storeFile (Almacenar).
 - ▶ Para descargar usamos retrieveFile (Recuperar).

```
private static boolean descargarFicheroASCII(FTPClient ftp, String directorioFichero, 1 us
        String nombreFichero) throws IOException {
    ftp.setFileType(FTP.ASCII_FILE_TYPE);
    // ruta y nombre en el que vamos a guardar el fichero.
    File fichero = new File(pathname: directorioFichero + "/" + nombreFichero);
    // El fichero en el remoto se llama igual que el nombre que le voy a poner en local.
    String ficheroRemoto = nombreFichero;
    OutputStream output = new FileOutputStream(fichero);
    boolean download = ftp.retrieveFile(ficheroRemoto, output);
    output.close();
    return download;
}
```

```
private static void pruebaDescargarFichero(){ 1 usage
    try {
        FTPClient ftp = conexion();

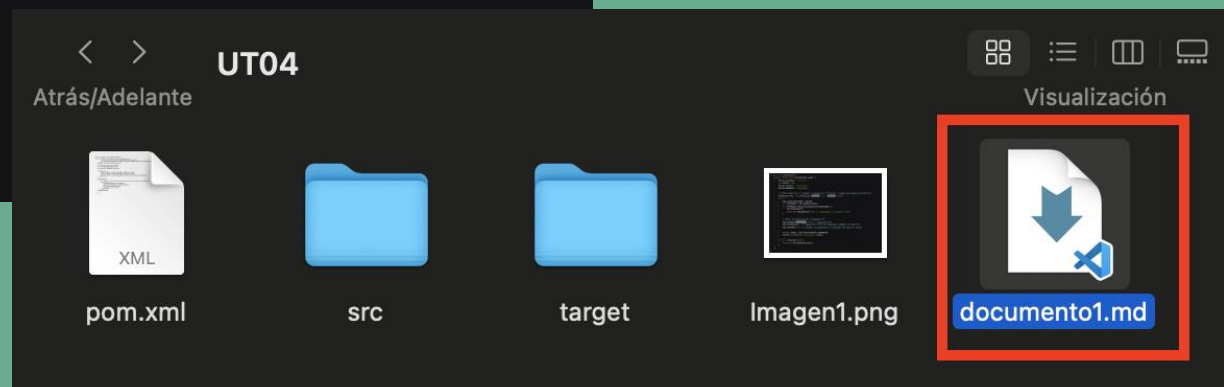
        if (ftp != null) {
            // Cambiamos a la ruta virtual del usuario guillermo.
            ftp.changeWorkingDirectory(path: "/datosFTP");

            // Obtener el directorio de trabajo local actual
            String directorioLocal = System.getProperty("user.dir");
            String nombreFichero = "documento1.md";

            boolean resultado = descargarFicheroASCII(ftp, directorioLocal, nombreFichero);
            System.out.println("¿Se ha descargado el fichero?" + resultado);

            ftp.logout();
            ftp.disconnect();
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

¿Se ha descargado el fichero?true





► Descarga tipo Binario

- Vamos a descargar un fichero cab.py que realmente es un fichero de texto, pero por no copiar un nuevo fichero en el servidor remoto.
- Al igual que vimos con la subida, la principal novedad es que dejamos de usar `retrieveFile` (que lo hace todo automáticamente) para usar `retrieveFileStream`
 - Ahora se crea un buffer de 4096 bytes para controlar manualmente la descarga con un bucle `while`
- Al igual que con la subida de los ficheros binarios debemos hacer uso del método `completePendingCommand()` y así esperar la confirmación final del servidor FTP para cerrar la transacción.

```
private static boolean descargarFicheroBinario(FTPClient ftp, String directorioFichero, 1
                                                String nombreFichero) throws IOException {

    ftp.setFileType(FTP.BINARY_FILE_TYPE);
    // ruta y nombre en el que vamos a guardar el fichero.
    File fichero = new File(pathname: directorioFichero + "/" + nombreFichero);
    // El fichero en el remoto se llama igual que el nombre que le voy a poner en local.
    String ficheroRemoto = nombreFichero;
    OutputStream output = new FileOutputStream(fichero);
    InputStream input = ftp.retrieveFileStream(ficheroRemoto);
    byte[] bytesIn = new byte[4096];
    int read = 0;

    while ((read = input.read(bytesIn)) != -1) {
        output.write(bytesIn, off: 0, read);
    }

    boolean download = ftp.completePendingCommand();
    input.close();
    output.close();
    return download;
}
```

```
private static void pruebaDescargarFichero(){ 1 usage
    try {
        FTPClient ftp = conexion();

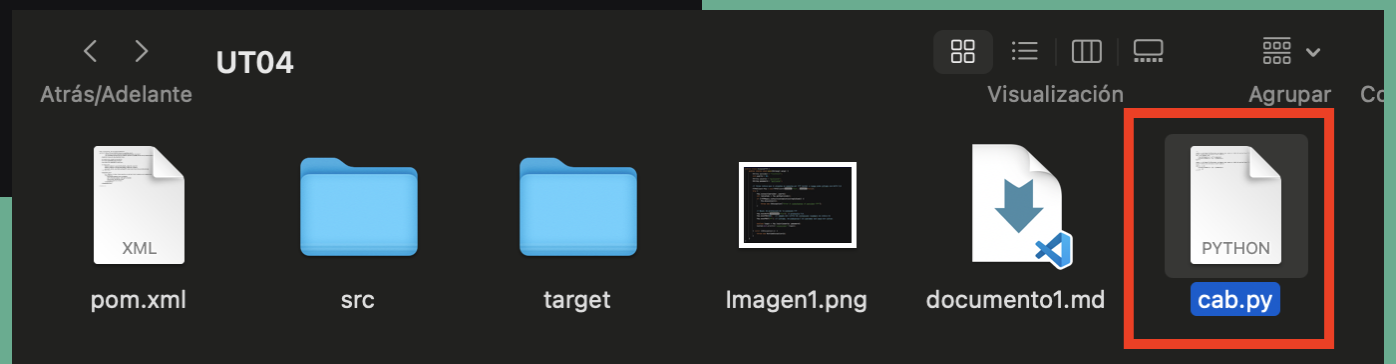
        if (ftp != null) {
            // Cambiamos a la ruta virtual del usuario guillermo.
            ftp.changeWorkingDirectory(path: "/datosFTP");

            // Obtener el directorio de trabajo local actual
            String directorioLocal = System.getProperty("user.dir");
            String nombreFichero = "cab.py";

            boolean resultado = descargarFicheroBinario(ftp, directorioLocal, nombreFichero);
            System.out.println("¿Se ha descargado el fichero?" + resultado);

            ftp.logout();
            ftp.disconnect();
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

Login: true
¿Se ha descargado el fichero?true



1.8. Otras acciones

- Mostrar información sobre los ficheros (o directorios) del directorio remoto.

```
private static void mostrarInformacion(FTPClient ftp) throws IOException {  
    FTPFile[] files = ftp.listFiles();  
    for (FTPFile fichero : files) {  
        System.out.println(fichero.getName());  
        System.out.println("\t"+fichero.getTimestamp().getTime());  
        System.out.println("\t"+fichero.getSize());  
        if (fichero.getType() == FTPFile.FILE_TYPE) {  
            System.out.println("\tFichero");  
        }  
        if (fichero.getType() == FTPFile.DIRECTORY_TYPE) {  
            System.out.println("\tDirectorio");  
        }  
    }  
}
```

► Crear directorios - Eliminar ficheros y directorios FTP remotos

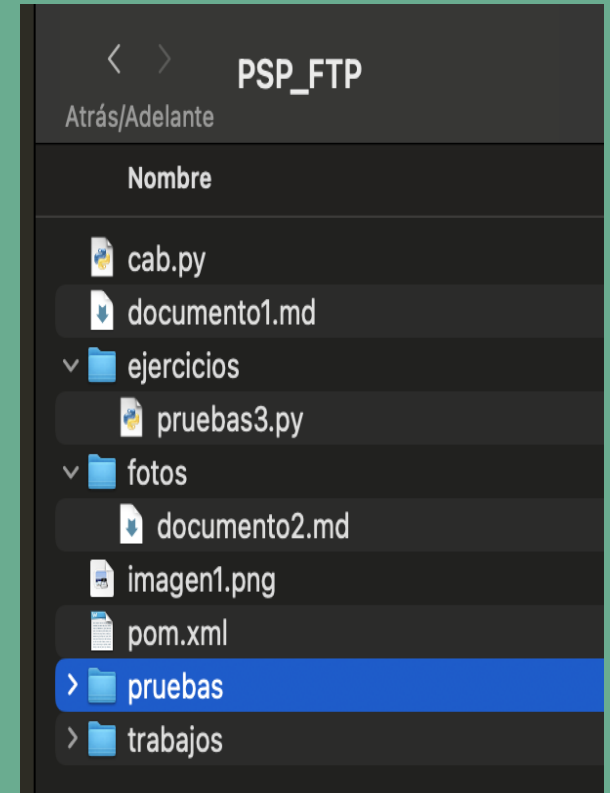
```
FTPClient ftp = conexion();

if (ftp != null) {
    // Cambiamos a la ruta virtual del usuario guillermo.
    ftp.changeWorkingDirectory(path: "/datosFTP");

    ftp.makeDirectory(path: "pruebas"); // Creamos el directorio pruebas

    /* Vamos a borrar el directorio examen, pero antes borramos todos los ficheros que tiene
    Este código funciona porque examen no tiene directorio, sino deberíamos hacer un método de
    borrado recursivo de estos*/
    String carpetaBorrar = "examen";
    FTPFile[] archivos = ftp.listFiles(carpetaBorrar);
    if (archivos != null && archivos.length > 0) {
        for (FTPFile archivo : archivos) {
            String rutaCompleta = carpetaBorrar + "/" + archivo.getName();
            ftp.deleteFile(rutaCompleta); // Borra un fichero
        }
    }
    ftp.removeDirectory(carpetaBorrar); // Borra un directorio (debe estar vacío)

    ftp.logout();
    ftp.disconnect();
}
```

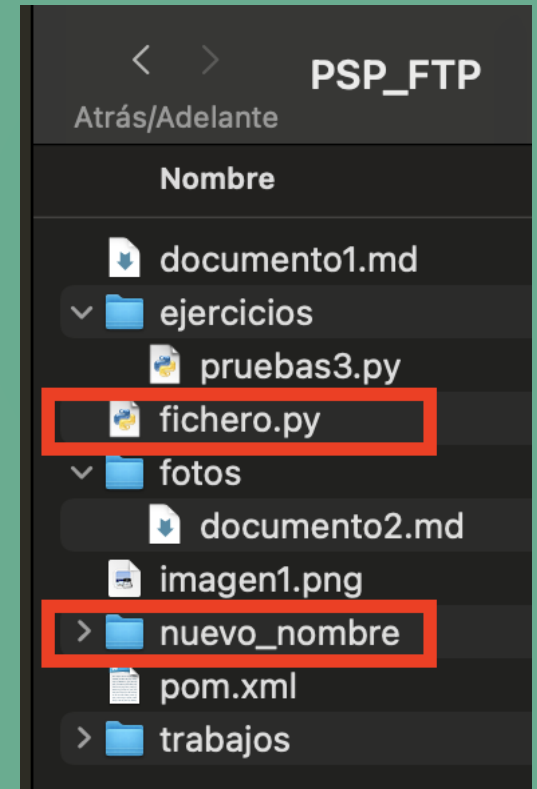


► Renombrar un fichero/directorio

```
try {
    FTPClient ftp = conexion();
    if (ftp != null) {
        // Cambiamos a la ruta virtual del usuario guillermo.
        ftp.changeWorkingDirectory(path: "/datosFTP");

        ftp.rename("pruebas", "nuevo_nombre"); // renombramos un directorio
        ftp.rename("cab.py", "fichero.py"); // renombramos un fichero.

        ftp.logout();
        ftp.disconnect();
    }
} catch (IOException e) {
    e.printStackTrace();
}
```



2. Correo

- ▶ Cuando buscamos en Internet cómo enviar correos electrónicos con Java, casi siempre aparecen dos nombres: JavaMail y Jakarta Mail → Realmente son la misma librería.
- ▶ ¿Por qué se encuentran con dos nombres? Durante muchos años, Java EE era la plataforma oficial para aplicaciones empresariales en Java. Más tarde, esas tecnologías pasaron a la Fundación Eclipse y Java EE cambió de nombre a Jakarta EE
 - ▶ Con ese cambio, JavaMail pasó a llamarse Jakarta Mail → El software es el mismo, solo cambió el nombre.
- ▶ La API de Jakarta Mail puede consultarse en:
<https://jakarta.ee/specifications/mail/2.1/jakarta-mail-spec-2.1> y define qué métodos existen.
 - ▶ Una API es un conjunto de interfaces y clases, dicen lo que se puede hacer, pero realmente no contiene el código real que lo hace.

- ▶ Por tanto, además de la API necesitamos una implementación de la misma que es la librería que cumple con la API y que contiene el código real.
 - ▶ En este caso entre otros contiene la conexión SMTP, autenticación, TLS/SSL envío del mensaje, etc.
- ▶ La API es el contrato, la implementación es quien lo cumple. Ejemplos de implementaciones de Jakarta Mail:
 - ▶ Angus Mail (Eclipse)
 - ▶ Oracle / com.sun.mail
- ▶ Si trabajas con proyectos Maven, existen dos opciones, añadir únicamente en el mismo la implementación o incluir la API y la implementación.
- ▶ **IMPORTANTE:** Si quiero cambiar de implementación (de Angus Mail de Eclipse por Oracle/com.sun.mail) el código no debe ser cambiado, únicamente la referencia la dependencia en el pom.xml (y seguramente los import realizados)

2.1. Clases principales de Jakarta Mail

- ▶ A lo largo de este apartado vamos a diseccionar las principales clases e interfaces que tiene la API de correo de Jakarta para enviar, leer y realizar otras acciones con mensajes de correo electrónico, al igual que en un sistema de correo típico.
- ▶ De entre los paquetes del proyecto Jakarta, dos de los más utilizados son jakarta.mail y jakarta.mail.internet.
- ▶ El paquete jakarta.mail
 - ▶ Es el corazón de la API. Contiene las clases que definen cómo funciona un sistema de correo de forma genérica.
 - ▶ Se encarga de la infraestructura: establecer la sesión, conectar con el servidor y gestionar el transporte de los mensajes.
 - ▶ Es donde definimos "las reglas del juego" (protocolos, hosts, puertos).

- ▶ El paquete `jakarta.mail.internet`
 - ▶ Mientras que el paquete anterior es genérico, este es específico para los estándares de Internet.
 - ▶ Aquí es donde "vive" el contenido del correo.
 - ▶ Proporciona las clases para manejar mensajes bajo el estándar MIME (que permite que los correos tengan formato HTML, tildes, archivos adjuntos, etc.).
 - ▶ Es el paquete que usaremos para dar forma a lo que el usuario final recibe.
- ▶ Las principales clases que estudiaremos de estos paquetes son
 - ▶ `jakarta.mail.Session`
 - ▶ `jakarta.mail.Message`
 - ▶ `jakarta.mail.Address`
 - ▶ `jakarta.mail.Authenticator`
 - ▶ `jakarta.mail.Transport`

▶ jakarta.mail.session

- ▶ Es como el centro de mandos y primer paso obligado para cualquier programa que quiera gestionar correos electrónicos.
- ▶ No sirve para escribir el mensaje ni para enviarlo directamente, sino para configurar el entorno. Sus tareas principales son:
 - ▶ Almacenar la configuración: Recopila las propiedades (como el host del servidor SMTP, el puerto, si se requiere SSL, etc.).
 - ▶ Gestionar la autenticación: Se encarga de validar nuestras credenciales frente al servidor.
 - ▶ Fábrica de conexiones: Es la que "fabrica" los objetos necesarios para transportar (Transport) o almacenar (Store) los correos.
- ▶ A diferencia de otras clases, no usamos `new Session()`. La API nos ofrece dos métodos estáticos para obtenerla, dependiendo de nuestras necesidades:
 - ▶ `getDefaultInstance()` devuelve una sesión "compartida" (la predeterminada). Es útil si solo necesitamos una configuración de correo en toda nuestra aplicación.
 - ▶ **`getInstance()`**: Crea y devuelve una **nueva sesión** independiente. Es la opción recomendada si nuestra aplicación necesita manejar diferentes configuraciones de correo simultáneamente (por ejemplo, enviar desde dos cuentas de distintos servidores).

► jakarta.mail.message

- Esta clase abstracta es el sobre y la carta en sí. Define cómo debe ser un mensaje, pero deja que sus subclases implementen los detalles técnicos.
- En la práctica, la subclase que utilizaremos siempre es MimeMessage (del paquete jakarta.mail.internet). Esta clase permite que nuestros correos cumplan con el estándar MIME, lo que nos permite enviar no solo texto plano, sino también HTML, caracteres especiales (tildes, eñes) y archivos adjuntos.
- Para poder redactar un correo (cargar datos en el objeto MimeMessage), utilizaremos los siguientes métodos fundamentales:
 - setFrom(Address) → Indica quién envía el correo (el campo De:)
 - setRecipients(tipo, dirección) → Define a los destinatarios.
 - El tipo puede ser: TO (Destinatario principal), CC: Copia de cortesía, BCC: Copia oculta (CCO).
 - setSubject(String) Establece el Asunto del mensaje.
 - setText(String) Define el cuerpo del mensaje como texto plano. Es el método más sencillo si no necesitamos formato.
 - setContent(obj, tipo) Es mucho más potente. Permite enviar contenido con formato, por ejemplo, HTML usando el tipo "text/html".
- Para usar estos métodos, el objeto MimeMessage necesita conocer la Session creada anteriormente → Message mensaje = new MimeMessage(sesion);.

► jakarta.mail.Address

- Cuando escribimos un correo, debemos indicar quién lo envía y quién lo recibe.
- Aunque nosotros solemos pensar en estas direcciones como simples cadenas de texto (como `alumno@alu.murciaeduca.es`), la API de Jakarta Mail utiliza la clase `Address` para gestionarlas de forma más profesional.
- Al igual que en el caso de `Message` trabajamos con la clase `MimeMessage`, en lugar de `Address` se utilizará la clase `InternetAddress` (`jakarta.mail.internet`).
 - Esta clase es muy útil porque no solo guarda la dirección de correo, sino que también: Valida el formato: Comprueba que la dirección tenga una estructura correcta (usuario, arroba y dominio).
 - Soporta nombres descriptivos: Permite que en el correo aparezca un nombre amigable en lugar de solo la dirección (por ejemplo: Guillermo Palazon <guillermo.palazon@muricaeduca.es>).
 - Para crear una dirección y asignarla a nuestro mensaje, lo haríamos de la siguiente manera:
 - Crear la dirección → `Address direccion = new InternetAddress("profesor@ies.es");`
 - Asignarla al mensaje (por ejemplo, al remitente) → `mensaje.setFrom(direccion);`

▶ jakarta.mail.Authenticator

- ▶ Para enviar un correo hoy en día, no basta con decir quiénes somos; hay que demostrarlo. La clase Authenticator es la encargada de gestionar ese proceso de "identificación" ante el servidor de correo.
- ▶ Como es una clase abstracta, nosotros solemos utilizar su implementación más común: PasswordAuthentication y tendremos que sobrescribir su método (creando una clase o usando una clase anónima) getPasswordAuthentication
- ▶ Normalmente, al crear la Session, le pasamos un objeto de este tipo. Es como darle al programa las llaves de nuestra cuenta para que pueda entrar y enviar correos en nuestro nombre.

▶ jakarta.mail.Transport

- ▶ Si la Session es el centro de mando y el Message es la carta, la clase Transport es el camión de reparto.
- ▶ Es la clase encargada de llevar el mensaje desde nuestra aplicación hasta el servidor del destinatario.
- ▶ Es una clase abstracta, la implementación real depende del protocolo que usemos (aunque SMTP es el estándar absoluto para envío).
- ▶ Aunque tiene métodos para un control detallado, la forma más sencilla y habitual de usarla es mediante su método estático send() que se encarga de abrir la conexión con el servidor (usando los datos de la sesión), enviar el mensaje y cerrar la conexión.

- ▶ Como ya se ha indicado, para poder utilizar todas estas clases se debe añadir una implementación de jakarta.mail en el pom.xml
- ▶ En mi caso vamos a utilizar la implementación de Eclipse

```
<dependency>  
  <groupId>org.eclipse.angus</groupId>  
  <artifactId>angus-mail</artifactId>  
  <version>2.0.2</version>  
  <type>jar</type>  
</dependency>
```


2.2. Envío correos electrónicos básicos

- ▶ Para poder enviar correos electrónicos básicos vamos a seguir una serie de pasos.
- ▶ 1. En primer lugar vamos a crear un objeto properties en el que vamos a tener la configuración del servidor SMTP del proveedor de servicios de correo electrónico. Vamos a tener que indicar:
 - ▶ mail.smtp.username – Nombre de usuario para acceder al servidor SMTP
 - ▶ mail.smtp.password – Contraseña de acceso al servidor SMTP
 - ▶ mail.smtp.host – Host del servidor SMTP
 - ▶ mail.smtp.port – Puerto mail.
 - ▶ smtp.auth – Se requiere autenticación.
 - ▶ mail.smtp.starttls.enable – TLS habilitado o no.
 - ▶ Estos valores, como el usuario y el password pueden estar almacenados en variables de entorno

► 2. Creación de la Sesión y Login (Session + Authenticator)

- Creamos el objeto Session llamando a getInstance().
- Importante: En este paso debemos registrar nuestro Authenticator que contiene el usuario y la contraseña. Es el momento en que la aplicación "pide permiso" al servidor para operar.
 - Jakarta Mail no permite pasar directamente usuario y contraseña; es obligatorio usar un Authenticator que devuelva un PasswordAuthentication cuando el sistema lo necesita.

1

```
private static Properties datosServidor(){ 1 usage
    final Properties prop = new Properties();
    prop.put("mail.smtp.username", System.getenv(name: "MAIL_USERNAME"));
    prop.put("mail.smtp.password", System.getenv(name: "MAIL_PASSWORD"));
    prop.put("mail.smtp.host", "smtp.gmail.com");
    prop.put("mail.smtp.port", "587");
    prop.put("mail.smtp.auth", "true");
    prop.put("mail.smtp.starttls.enable", "true");
    return prop;
}
```

2

```
private static Session crearSession(Properties prop){ no usages
    // Clase anónima que extiende Authenticator y devuelve las credenciales cuando
    // el servidor de correo las solicita
    Authenticator auth = new Authenticator() {
        @Override no usages
        protected PasswordAuthentication getPasswordAuthentication() {
            return new PasswordAuthentication(
                prop.getProperty("mail.smtp.username"),
                prop.getProperty("mail.smtp.password")
            );
        }
    };
    // Se crea la sesión de correo usando la configuración y el Authenticator
    // para la autenticación SMTP
    Session mailSession = Session.getInstance(prop, auth);
    return mailSession;
}
```

- ▶ 3. Empezamos a crear el mail que se enviará.
 - ▶ Para ello creamos un objeto de tipo `MimeMessage` a partir del objeto `Session`.
 - ▶ Para este mensaje establecemos
 - ▶ `setFrom`: ¿Quién lo envía? `setRecipients`:
 - ▶ ¿A quién va dirigido? (Podemos usar TO, CC o BCC).
 - ▶ `setSubject`: El título que verá el usuario en su bandeja de entrada.
 - ▶ Utilizar el método `setText()` para establecer el contenido del mensaje de correo electrónico.
- ▶ 4. Una vez el mensaje está listo, se lo entregamos a la clase `Transport`. Al llamar a `Transport.send(mensaje)`, la librería abre la conexión, entrega el paquete al servidor SMTP y cierra la conexión.
- ▶ 5. Todo este proceso debe estar envuelto en un bloque try-catch. La excepción principal es `MessagingException`. Capturarla es vital para saber si el error fue por una contraseña incorrecta, un servidor caído o una dirección de correo mal escrita.

```
private static void envioMail(Properties prop, Session mailSession){ 1 usage
    try {
        Message mail = new MimeMessage(mailSession);
        mail.setFrom(new InternetAddress(prop.getProperty("mail.smtp.username")));
        mail.setSubject("Asunto de e-mail de prueba ");
        // Indicamos destinatarios y los establecemos
        InternetAddress[] toEmail =
            InternetAddress.parse(addresslist: "guillermo.palazon@murciaeduca.es, otrousuario1@gmail.com");
        InternetAddress[] ccEmail =
            InternetAddress.parse(addresslist: "otrousuario2@gmail.com, otrousuario3@gmail.com");
        InternetAddress[] bccEmail =
            InternetAddress.parse(addresslist: "otrousuario4@gmail.com");
        mail.setRecipients(Message.RecipientType.TO, toEmail);
        mail.setRecipients(Message.RecipientType.CC, ccEmail);
        mail.setRecipients(Message.RecipientType.BCC, bccEmail);
        // Indicamos el texto del mail
        mail.setText("Hola,\n\n\tPrueba de e-mail.");
        //Enviamos e-mail.
        Transport.send(mail);
    } catch (MessagingException e) {
        throw new RuntimeException(e);
    }
}
```

3

4

5

```
public static void main(String[] args) {
    Properties prop = datosServidor();
    Session sessionMail = crearSession(prop);
    envioMail(prop, sessionMail);
}
```

- ▶ Sin embargo, aunque todo el código, además del usuario y la contraseña está perfecto, no nos envía el e-mail y da error.

```
Exception in thread "main" java.lang.RuntimeException: jakarta.mail.AuthenticationFailedException: 535-5.7.8 Username and Password not accepted. For more information, go to
```

```
535 5.7.8 https://support.google.com/mail/?p=BadCredentials 5b1f17b1804b1-48066beeaf9sm361528185e9.6 - gsmt
```

```
at org.example.correo.EnvioMail.envioMail(EnvioMail.java:65)
```

```
at org.example.correo.EnvioMail.main(EnvioMail.java:13)
```

```
Caused by: jakarta.mail.AuthenticationFailedException: 535-5.7.8 Username and Password not accepted. For more information, go to
```

```
535 5.7.8 https://support.google.com/mail/?p=BadCredentials 5b1f17b1804b1-48066beeaf9sm361528185e9.6 - gsmt
```

```
at org.eclipse.angus.mail.smtp.SMTPTransport$Authenticator.authenticate(SMTPTransport.java:954)
```

```
at org.eclipse.angus.mail.smtp.SMTPTransport.authenticate(SMTPTransport.java:865)
```

```
at org.eclipse.angus.mail.smtp.SMTPTransport.protocolConnect(SMTPTransport.java:769)
```

```
at jakarta.mail.Service.connect(Service.java:367)
```

```
at jakarta.mail.Service.connect(Service.java:225)
```

```
at jakarta.mail.Service.connect(Service.java:174)
```

```
at jakarta.mail.Transport.send0(Transport.java:232)
```

```
at jakarta.mail.Transport.send(Transport.java:102)
```

```
> <2 folded frames>
```

- ▶ El error es claro: Código 535: Es el código SMTP estándar para errores de autenticación. Causa: Google ha recibido el usuario, pero no acepta la clave proporcionada.
- ▶ El problema es que Google bloquea el acceso por seguridad si intentas usar tu contraseña normal de Gmail.
- ▶ Esta aplicación no permite el acceso mediante "aplicaciones menos seguras" usando la contraseña principal.
 - ▶ La solución: Hay que activar la Verificación en dos pasos en la cuenta de Google y generar una "Contraseña de aplicación" específica de 16 caracteres.
 - ▶ Esa es la que debes poner en tu código (o variable de entorno), no tu contraseña de usuario

← Verificación en dos pasos



Llaves de acceso y llaves de seguridad



Añadir una llave de seguridad



Notificación de Google



2 dispositivos



Authenticator



Añadir aplicación Authenticator



Número de teléfono



646 14 64 25



Códigos de verificación alternativos



10 códigos disponibles



Contraseñas de aplicación

Las contraseñas de aplicación no son recomendables y, en la mayoría de los casos, son innecesarias. Para proteger tu cuenta, usa Iniciar sesión con Google para conectar aplicaciones a tu cuenta de Google.

Contraseñas de aplicación

1 contraseña de aplicación



← Contraseñas de aplicación

Las contraseñas de
Google en aplicaci
estándares de segu

Las contraseñas de
servicios actualizac
Antes de crear una
aplicación la necesi
[Más información](#)

Tus contraseñas

Jakarta Mail

Para crear una con
a continuación...

Nombre de la

Contraseña de aplicación generada

Tu contraseña de aplicación para el dispositivo

tl[REDACTED]m

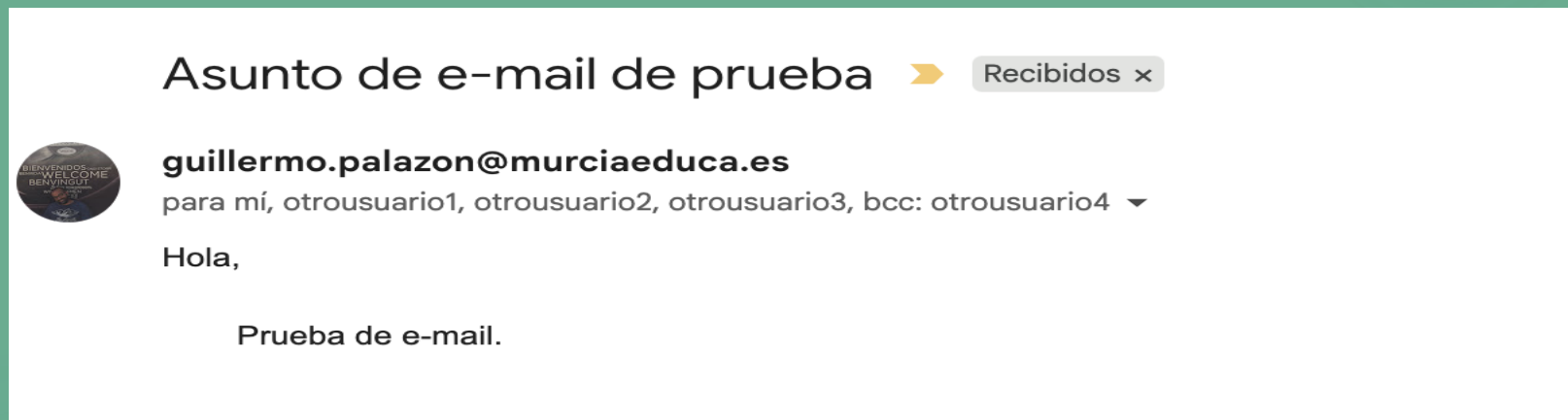
Cómo utilizarla

Accede a la sección de configuración de tu cuenta de Google en la aplicación o el dispositivo que estés intentando configurar. Sustituye tu contraseña por la contraseña de 16 caracteres que se muestra arriba.

Al igual que la contraseña normal, esta contraseña de aplicación ofrece acceso completo a tu cuenta de Google. No tendrás que recordarla, así que no la escribas ni la compartas con nadie.

[Hecho](#)

- ▶ Se vuelve a enviar el nuevamente y se comprueba que llega



- ▶ Aunque el texto sin formato (setText) es útil para pruebas rápidas, en el mundo real la mayoría de los correos utilizan HTML.
 - ▶ Esto nos permite usar negritas, colores, tablas e incluso imágenes, mejorando enormemente la usabilidad y el aspecto visual.
- ▶ Para enviar un correo HTML en Jakarta Mail, solo tenemos que hacer un pequeño cambio: sustituimos el método setText() por el método setContent(), especificando el tipo MIME adecuado.

```
// mail.setText("Hola,\n\n\tPrueba de e-mail.");
String textoHTML = ""
    <div style='font-family: Arial, sans-serif; border: 1px solid #2c3e50; padding: 20px; border-radius: 10px;'>
    <h1 style='color: #2980b9;'>¡¡¡Hola!!!</h1>
    <p>Este es un <strong>mensaje de prueba</strong> enviado desde Java (API <strong>Jakarta Mail</strong>).</p>
    <hr>
    <p style='font-size: 0.9em; color: #7f8c8d;'>
        Saludos,<br>
        <strong>Guillermo Palazón Cano</strong><br></p></div>"";

// Establecemos el contenido indicando que es HTML
mail.setContent(textoHTML, S: "text/html; charset=utf-8");
```



guillermo.palazon@murciaeduca.es

para mí, otrousuario1, otrousuario2, otrousuario3, bcc: otrousuario4 ▼

¡¡¡Hola!!!

Este es un **mensaje de prueba** enviado desde Java (API **Jakarta Mail**).

Saludos,
Guillermo Palazón Cano

2.3. Enviar correos complejos

- ▶ Para enviar archivos adjuntos (como PDFs, imágenes o archivos ZIP), ya no podemos tratar el mensaje como una simple cadena de texto. Necesitamos pasar al concepto de Mensaje Compuesto o Multipart.
- ▶ Cuando un correo lleva adjuntos, en realidad estamos enviando varias piezas unidas en un solo paquete. Para gestionar esto, Jakarta Mail utiliza un sistema de "piezas" (BodyParts) y un "contenedor" (Multipart).
- ▶ **1. MimeMultipart.** Imagina que el correo es una caja. El objeto MimeMultipart es esa caja que sirve de envoltorio (*wrapper*) para agrupar todas las partes del mensaje y que viajen juntas

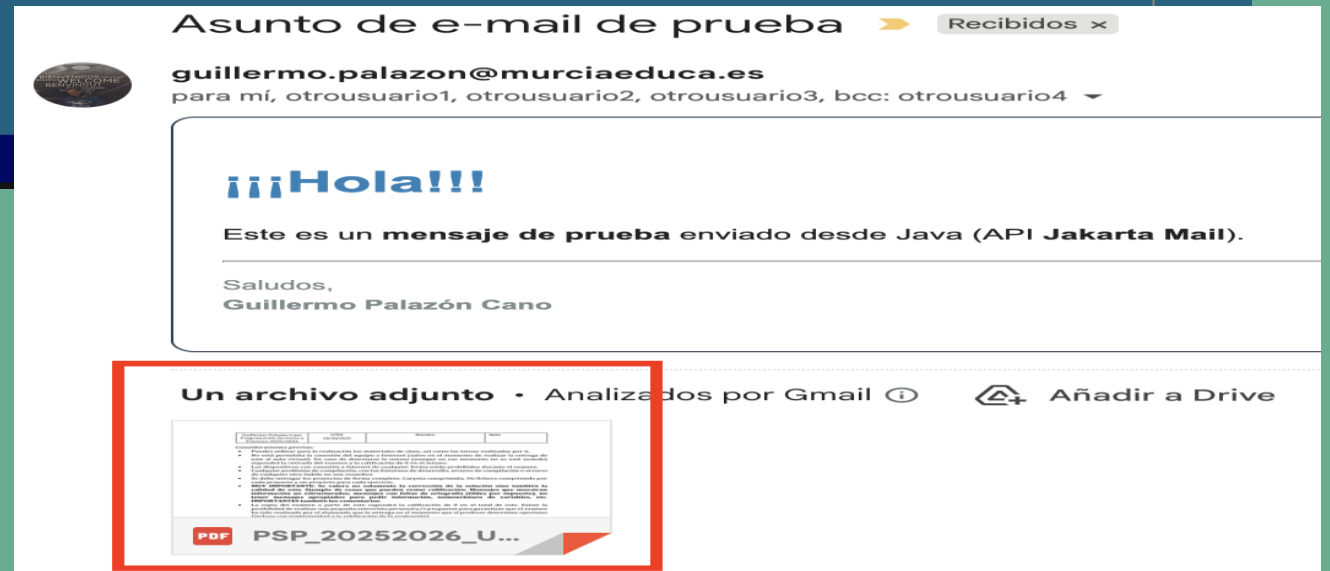
- 
- ▶ **2. MimeBodyPart.** Cada elemento del correo es una pieza independiente que introducimos en la caja:
 - ▶ Parte 1 (El texto): Creamos una MimeBodyPart para el cuerpo del mensaje (ya sea texto plano o HTML).
 - ▶ Parte 2 y sucesivas (Los adjuntos): Creamos una nueva MimeBodyPart por cada archivo que queramos enviar.
 - ▶ **3. Proceso para envío del mensaje**
 - ▶ Creamos el contenedor: `MimeMultipart contenedor = new MimeMultipart();`
 - ▶ Añadimos el texto: Creamos la parte del texto y la añadimos al contenedor con `addBodyPart()`.
 - ▶ Añadimos el archivo:
 - ▶ Cargamos el archivo físico mediante un `FileDataSource` y lo configuramos en una nueva pieza y la añadimos al contenedor
 - ▶ Realmente la subida se puede hacer con el método `attachFile(ruta_fichero)`.
 - ▶ Cerramos el sobre: Le decimos al mensaje principal que su contenido es ese contenedor: `message.setContent(contenedor);`
 - ▶ Enviamos: Como siempre, usamos `Transport.send(message);`.

```
String textoHTML = ""
<div style='font-family: Arial, sans-serif; border: 1px solid #2c3e50; padding: 20px; border-radius: 10px;'>
<h1 style='color: #2980b9;'>¡¡¡Hola!!!</h1>
<p>Este es un <strong>mensaje de prueba</strong> enviado desde Java (API <strong>Jakarta Mail</strong>).</p>
<hr>
<p style='font-size: 0.9em; color: #7f8c8d;'>
    Saludos,<br>
    <strong>Guillermo Palazón Cano</strong><br></p></div>"";
```

```
MimeBodyPart parteTexto = new MimeBodyPart();
parteTexto.setContent(textoHTML, type: "text/html; charset=utf-8");
```

```
MimeBodyPart parteAdjunto = new MimeBodyPart();
parteAdjunto.attachFile("/Users/guillermopalazoncano/Downloads/PSP_20252026_UT01.pdf");
```

```
Multipart multipart = new Multipart();
multipart.addBodyPart(parteTexto);
multipart.addBodyPart(parteAdjunto);
mail.setContent(multipart);
```



2.4. Lectura de emails con Jakarta Mail

- ▶ Si para enviar usábamos un "camión de transporte" (Transport), para leer necesitamos un "almacén". La clase **Store** (`jakarta.mail.Store`) es la encargada de conectar con el servidor de correo entrante y darnos acceso a las carpetas y mensajes guardados.
- ▶ Dependiendo de cómo queramos gestionar los mensajes, el Store utilizará un protocolo u otro:
 - ▶ **POP3Store**: Para descargar los mensajes al ordenador local (normalmente se borran del servidor).
 - ▶ **IMAPStore**: Estándar actual. Permite ver los mensajes que están en el servidor sin borrarlos, gestionar carpetas y sincronizar estados (leído/no leído). Será el que utilizemos en este apartado.

- ▶ Leer el buzón de entrada (IMAP) requiere un proceso más estructurado que el envío, ya que debemos navegar por la jerarquía de carpetas del servidor:
 - ▶ **Sesión (Session):** Al igual que en el envío, configuramos las propiedades (host IMAP, puerto 993 para SSL, etc.).
 - ▶ **Conexión:** Abrimos la conexión real con el servidor usando el usuario y la contraseña.
 - ▶ **Obtención del Store:** Le pedimos a la sesión que nos dé un objeto para el protocolo "imaps".
 - ▶ **Localizar la Carpeta:** Los correos no están "suelos", están en carpetas. Normalmente buscaremos la carpeta llamada **"INBOX"**.
 - ▶ **Apertura:** Debemos abrir la carpeta en modo lectura (READ_ONLY) o lectura/escritura (READ_WRITE).
 - ▶ **Recuperación:** Obtenemos un array de objetos Message[].
 - ▶ **Procesamiento:** Recorremos el array para leer el asunto, el remitente o el cuerpo de cada correo.
 - ▶ **Cierre de Carpeta/Store:** Es vital cerrar la carpeta para liberar recursos en el servidor.
 - ▶ **Desconexión:** Cerramos la sesión y la conexión física.


```
private static Properties datosServidorImap() { 1 usage
    final Properties prop = new Properties();
    prop.put("mail.imap.username", System.getenv(name: "MAIL_USERNAME"));
    prop.put("mail.imap.password", System.getenv(name: "MAIL_PASSWORD"));
    prop.put("mail.imap.host", "imap.gmail.com");
    prop.put("mail.imap.port", "993");
    prop.put("mail.imap.auth", "true");
    prop.put("mail.imap.ssl.enable", "true");
    return prop;
}
```

```
private static Session crearSessionImap(Properties prop) { 1 usage
    // Clase anónima que extiende Authenticator y devuelve las credenciales cuando
    // el servidor de correo las solicita
    Authenticator auth = new Authenticator() {
        @Override no usages
        protected PasswordAuthentication getPasswordAuthentication() {
            return new PasswordAuthentication(
                prop.getProperty("mail.imap.username"),
                prop.getProperty("mail.imap.password")
            );
        }
    };
    Session mailSession = Session.getInstance(prop, auth);
    return mailSession;
}
```

```

try {
    Properties prop = datosServidorImap();
    Session sessionMail = crearSessionImap(prop);
    // Obtenemos el Store
    Store store = sessionMail.getStore(protocol: "imap");
    store.connect(prop.getProperty("mail.imap.host"),
        prop.getProperty("mail.imap.username"),
        prop.getProperty("mail.imap.password"));
    // Obtenemos la carpeta
    Folder carpeta = store.getFolder(s: "INBOX");
    carpeta.open(Folder.READ_ONLY);
    // Obtenemos los mensajes
    Message[] mailsRecibidos = carpeta.getMessages();

    // Leemos como máximo 3 mensajes (más antiguos)
    int numMailMostrar = Math.min(mailsRecibidos.length, 3);
    for (int i = 0; i < numMailMostrar; i++) {
        System.out.println("Mail número " + (i + 1));
        Message mailRecibido = mailsRecibidos[i];
        System.out.println("\tRemitente: " + mailRecibido.getFrom()[0]);
        System.out.println("\tAsunto: " + mailRecibido.getSubject());
        System.out.println("\tFecha: " + mailRecibido.getSentDate());
    }
} catch (MessagingException e) {
    throw new RuntimeException(e);
}

```

Mail número 1

Remitente: Equipo de Gmail <mail-noreply@google.com>

Asunto: Cómo utilizar Gmail con Google Apps

Fecha: Wed Nov 04 00:19:00 CET 2015

Mail número 2

Remitente: Equipo de Gmail <mail-noreply@google.com>

Asunto: Lo mejor de Gmail estás donde estás

Fecha: Wed Nov 04 00:19:01 CET 2015

Mail número 3

Remitente: Equipo de Gmail <mail-noreply@google.com>

Asunto: Consejos para utilizar tu nueva bandeja de entrada

Fecha: Wed Nov 04 00:19:01 CET 2015

3. Comunicación mediante HTTP

- ▶ Hasta la versión 8 de Java, la clase `URLConnection` del paquete `java.net` ha sido la base de aplicaciones capaces de acceder a recursos de la web.
- ▶ En java 11 se introdujo el paquete `java.net.http` que proporciona una alternativa más potente, sencilla y actualizada de realizar peticiones HTTP.
- ▶ Para realizar peticiones HTTP convencionales se utilizan principalmente tres clases:
 - ▶ `HttpClient` → Cliente HTTP
 - ▶ `HttpRequest` → Petición
 - ▶ `HttpResponse` → Respuesta

- ▶ Para realizar una petición HTTP se deben realizar los siguientes pasos:
 - ▶ Crear el objeto HttpClient, indicando versión del protocolo, así como otros datos opcionales como el comportamiento en caso de que existan redirecciones del servidor.
 - ▶ Crear el objeto HttpRequest, indicando la URI y los parámetros de la cabecera de la petición.
 - ▶ Realizar la petición a través del método send de HttpClient y asignar la respuesta de la petición al objeto HttpResponse.
 - ▶ Procesar la petición.
- ▶ Ejemplo 1. Vamos a utilizar una API Rest que nos devuelve frases graciosas de Chuck Norris.

```
try {
    // 1. Creamos cliente HTTP
    HttpClient client = HttpClient.newHttpClient();

    // 2. Creamos petición
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create("https://api.chucknorris.io/jokes/random"))
        .GET()
        .build();

    // 3. Enviar petición y recibir respuesta.
    // Envía la petición HTTP usando el cliente, espera la respuesta del servidor, guarda esa
    // respuesta en un objeto HttpResponse y convierte el contenido recibido en texto (String)

    HttpResponse<String> response =
        client.send(request, HttpResponse.BodyHandlers.ofString());

    System.out.println("Código HTTP: " + response.statusCode());
    System.out.println("Respuesta:");
    System.out.println(response.body());
} catch (Exception e) {
    e.printStackTrace();
}
```

Código HTTP: 200

Respuesta:

```
{"categories":[],"created_at":"2020-01-05 13:42:28.664997","icon_url":"https://api.chucknorris.io/img/avatar/chuck-norris.png","id":"__5gqMxySZCkPRpbbR0y3g",
"updated_at":"2020-01-05 13:42:28.664997","url":"https://api.chucknorris.io/jokes/_5gqMxySZCkPRpbbR0y3g","value":"The reason why Chuck Norris is such a badass is
he thought all the yo momma jokes were written about his mother."}
```

► https://api.chucknorris.io/jokes/LOkwmSLrR1CE8L_HRYjwjg



Prev

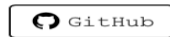
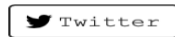
Next

// They say lightning never strikes in the same place twice - tell that to last guy to receive a one,two punch from Chuck Norris.

You can use the left and right keys on your keyboard to navigate!



Chucknorris.io is free and will always be!
However, as maintaining this service costs \$\$\$,
we are glad to be sponsored by Jugendstil.io.



Attribution: Artwork "Dancing Chuck" by [jesgrad07](#)

Legal disclaimer: This website and its creators are not affiliated with Chuck Norris, any motion picture corporation, any television corporation, parent, or affiliate corporation. All motion pictures, products, and brands mentioned on this website are the respective trademarks and

- ▶ Ejemplo 2. Vamos a descargar una página (debe ser HTTP) y guardarla en un fichero HTML en local del proyecto.
- ▶ La página una vez que la abramos en el navegador es posible que no se vea exactamente igual ya que pierda algunos estilos.

```
HttpClient client = HttpClient.newHttpClient();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("http://www.edu4java.com/es/web/web30.html"))
    .GET()
    .build();

HttpResponse<String> response =
    client.send(request, HttpResponse.BodyHandlers.ofString());

// 4. Copio la respuesta en un fichero.
File fichero = new File(pathname: "pagina.html");
FileWriter fw = new FileWriter(fichero);
fw.write(response.body());
fw.close();
```

<> pagina.html x

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
2
3 <html xmlns="http://www.w3.org/1999/xhtml" lang="es"><!-- InstanceBegin template="/Templates/es-html-css.dwt" codeOutsideHTMIsLocked="false"
4
5 <head>
6
7 <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />
8
9 <!-- InstanceBeginEditable name="doctitle" -->
10 <meta name="description" content="En este tutorial vamos a aprender sobre el Servidor Web o Servidor HTTP. El cliente es el Browser que so
11 <title>Servidor Web o Servidor HTTP</title>
12 <link rel="alternate" href="http://www.edu4java.com/en/web/web-server.html" hreflang="en" />
13 <link rel="alternate" href="http://www.edu4java.com/es/web/web30.html" hreflang="es" />
14 <!-- InstanceEndEditable -->
15
16
17
18 <link href="/css/prettify.css" rel="stylesheet" type="text/css" />
19
20 <link href="/css/edu4java1.css" rel="stylesheet" type="text/css" />
21
22 <script type="text/javascript">
23
24
25
26 var _gaq = _gaq || [];
27
28 _gaq.push(['_setAccount', 'UA-19105638-1']);
29
30 _gaq.push(['_trackPageview']);
```

MATERIALES DE CONSULTA

▶ Libros de Texto Recomendados

▶ Editorial Paraninfo.

- ▶ Programación de Servicios y Procesos. Fernando Paniagua Martín

- ▶ [Enlace al libro](#)

▶ Editorial McGraw Hill

- ▶ Programación de Servicios y Procesos. José Alfredo Murcia Andrés y Joan Gerard Camarena Estruch

- ▶ [Enlace al libro](#)

▶ URL

- ▶ Material Óscar Maestre → <https://oscarmaestre.github.io/servicios/index.html>

- ▶ Material Vicente Martínez → https://psp2dam.github.io/psp_pages/es/



▶ MUCHAS GRACIAS POR SU
ATENCIÓN

▶ ¿ALGUNA DUDA?

▶ guillermo.palazon@murciaeducaca.es